

Question:

Implement the following algorithms using Divide & Conquer approach.

1. Binary search to search for an element in a sorted array
2. min-max algorithm to find the minimum and maximum elements in a list

Analyze the time complexity of both algorithms

Introduction

The Divide and conquer approach is a fundamental paradigm in algorithm design that works by recursively breaking down a problem into two or more sub-problems of the same or related type. These sub-problems are solved independently and their solutions are combined to give a solution to the original problem.

Objective

- To understand the fundamental mechanism of the divide & conquer paradigm
- To implement the Binary search algorithm for efficient searching in sorted arrays
- To implement the min-max algorithm using a recursive approach to find limits in a list
- To analyze time and space complexity

Theory

Binary Search

It is an efficient algorithm to finding an item from a sorted list of items. It works by dividing a half the portion of the list until you have narrowed down the possible location to just one. The input array must be sorted.

Recurrence Relation: $T(n) = T(n/2) + c$

min-max algorithm:

Here, the array is divided into halves, finds the min and max of each half recursively and then compares the results to find the global maximum and minimum.

Recurrence Relation: $T(n) = 2T(n/2) + 2$

Algorithm

→ Binary Search

- i) calculate the middle index:
$$mid = (low + high) / 2$$
- ii) If $arr[mid]$ is equal to the target, return mid
- iii) If $target < arr[mid]$, recurse on left half
(low to $mid-1$)
- iv) If $target > arr[mid]$, recurse on right half
($mid+1$ to high)
- v) If the range is exhausted and the element is not found, return -1.

Min-Max

- i) If the array size is 1, return the element as both min & max
- ii) If the array size is 2, compare the two elements and return the min and max

- iii) otherwise, divide the array into two halves
- iv) recursively find min1, max1 for left half
and min2, max2 for the right half
- v) The global max is max(max1, max2) and
global min is min(min1, min2).

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct pair {
    int min;
    int max;
};

int binarySearch(int arr[], int low, int high, int x)
{
    if (high <= low)
    {
        int mid = (low + high) / 2;
        if (arr[mid] == x) return mid;
        if (arr[mid] > x) return binarySearch(arr, low, mid - 1, x);
        return binarySearch(arr, mid + 1, high, x);
    }
    return -1;
}

pair getMinMax(int arr[], int low, int high)
{
    pair minmax, left, right;
    int mid;
    if (low == high)
    {
        minmax.max = arr[low];
        minmax.min = arr[low];
        return minmax;
    }
    if (high == low + 1)
    {
        minmax.max = max(arr[low], arr[high]);
        minmax.min = min(arr[low], arr[high]);
        return minmax;
    }
    mid = (low + high) / 2;
    left = getMinMax(arr, low, mid);
    right = getMinMax(arr, mid + 1, high);
    minmax.max = max(left.max, right.max);
    minmax.min = min(left.min, right.min);
    return minmax;
}
```

```

1 if(arr[low] > arr[high])
{
    minmax.max = arr[low];
    minmax.min = arr[high];
} else {
    minmax.max = arr[high];
    minmax.min = arr[low];
}

return minmax;
}

mid = (low + high) / 2;
left = getMinMax(arr, low, mid);
right = getMinMax(arr, mid + 1, high);

minmax.min = (left.min < right.min) ? left.min : right.min;
minmax.max = (left.max > right.max) ? left.max : right.max;

return minmax;
}

int main()
{
    int arr[] = {3, 5, 8, 17, 24, 78};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 24;

    int res = binarySearch(arr, 0, n - 1, target);
    if (res + 1 == -1)
        cout << "Element is in" << res << endl;
    else
        cout << "Element not found" << endl;

    pair<int, int> p = getMinMax(arr, 0, n - 1);
    cout << "Minimum element: " << p.min << endl;
    cout << "Maximum element: " << p.max << endl;

    return 0;
}

```

Output

```

Element is in: 4
Minimum element: 3
Maximum element: 78

```


Time complexity analysis

Binary search: $O(\log n)$ Best case: $O(1)$

Min-max: $O(n)$ Best case: $O(n)$

Conclusion.

The binary search proved to be significantly faster than linear method for sorted dataset and min-max algorithm showed that structure decomposition can optimize the number of comparisons required to find boundaries in a list.

References

- DAA notes
- Hamro csit
- geckrothcees
- W3School

Quick sort

It picks an element as a 'pivot' and partitions the given array around the picked pivot.

Randomized Quicksort

A random element is chosen as the pivot to avoid the $O(n^2)$ worst-case scenario of deterministic quick sort.

Algorithm:

→ Merge Sort:

- (i) if low < high,
- (ii), find the middle point $\text{mid} = (\text{low} + \text{high}) / 2$
- (iii) call mergesort for the first half
 : (low, mid)
- (iv) call mergesort for the second half
 (mid+1, high)
- (v) merge the two halves using a temporary array

→ Quick Sort (1997-1017)

- (i) pick the last element as the pivot
- (ii) maintain an index i as the boundary for elements smaller than the pivot
- (iii) Traverse the array:
 - if $arr[j] < pivot$
 - swap $arr[i]$ and $arr[j]$
 - increment i
- (iv) Finally, swap the pivot into its correct sorted position at $i+1$
- (v) Recursively apply the process to the left and right partitions

```

Code:
#include<iostream>
#include<vector>
#include<stdlib.h>
#include<time.h>
using namespace std;
// merge
void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for(int i = 0; i < n1; i++) L[i] = arr[l + i];
    for(int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while(i < n1 && j < n2)
    {
        if(L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while(i < n1) arr[k++] = L[i++];
    while(j < n2) arr[k++] = R[j++];
}

void mergesort(int arr[], int l, int r)
{
    if(l < r)
    {
        int m = l + (r - l) / 2;
        mergesort(arr, l, m);
        mergesort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

// quicksort logic
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = arr[low - 1];
    for(j = low; j <= high - 1; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

```

```

void quicksort(int arr[], int low, int high,
               bool randomized)
{
    if (low < high)
    {
        int pi = randomized ? randomPartition(
            arr, low, high) :
            partition(arr, low, high);
        quicksort(arr, low, pi-1, randomized);
        quicksort(arr, pi+1, high, randomized);
    }
}

void printArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    srand(time(0));
    int data[] = {3, 4, 7, 5, 6};
    int n = 5;
    cout << "Original array";
    printArray(data, n);
    mergesort(data, 0, n-1);
    cout << "Merge sort result: ";
    printArray(data, n);
    int data2[] = {2, 5, 6, 4, 3, 2, 5};
    quicksort(data2, 0, 5, true);
    cout << "Randomized quicksort result: ";
    printArray(data2, 6);
    return 0;
}

```

Output

```

Original array: 3 4 7 5 6
Merge sort result: 3 4 5 6 7
Randomized quicksort result
1 2 3 4 5 6

```


Complexity Analysis

Algorithms	Base case	Average case	Worst case
merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Randomized sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ Avg

Conclusion.

The experiment confirmed that merge sort provides consistent $O(n \log n)$ performance at the cost of higher memory usage. Deterministic Q.S is highly efficient but vulnerable to worst case scenarios on sorted data was effectively mitigated by Randomized Q.S

References

- w3school
- USA notes
- claude

Question:

1. Implement algorithms related to heap data structures and order statistics.
2. Implementation of max heap or min heap including Build heap and heapify operations.
2. Implement the heap sort algorithm and analyze its complexity.
3. Find the k th smallest element using order statistics including Brute-force selection, expected linear time selection and worst case linear time selection.

Introduction

Heaps are specialized tree-based data structures that satisfy the heap property. They are essential for priority queues and efficient sorting.

Objective

To implement Heapify, Build Heap, Heapsort and various methods for order statistics (Finding k th smallest element).

Theory

→ Max-Heap

It is a complete binary tree where the value of each node is greater than or equal to its children.

→ Build Heap

Converts an unordered array into heap in $O(n \log n)$ time.

→ Heap sort

Builds a heap and repeatedly extracts the maximum element.

→ Order statistics

Finding the k th smallest element. Quick select provides linear time, while median of medians guarantees $O(n \log n)$ even in worst case.

Algorithm and code

```

def heapify(arr, n, i):
    largest = i
    l1 = 2 * i + 1, l2 = i + 2
    if l1 < n and arr[l1] > arr[largest]: largest = l1
    if l2 < n and arr[l2] > arr[largest]: largest = l2
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heapSort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

def quickSelect(arr, low, high, k):
    if low <= high:
        p1 = partition(arr, low, high)
        if p1 == k - 1:
            return arr[p1]
        return quickSelect(arr, low, p1 - 1, k)
    if p1 > k - 1:
        return quickSelect(arr, p1 + 1, high, k)

```

Output

- heapSort: [4, 20, 3, 5, 1] → [1, 3, 5, 20, 4]
- order statistic [12, 3, 5, 7, 28]
 - k = 2 → output = 5

Complexity Analysis

- Heapify: $O(\log n)$
- Build heap: $O(n)$
- Heap sort: $O(n \log n)$
- Quick select (Expected): $O(n)$
- median - or - medians (worst case): $O(n)$

Conclusion

Heaps provide an efficient $O(n \log n)$ sorting method with $O(1)$ auxiliary space. For selection problems, quick select is typically preferred due to its $O(n)$ avg performance.

References

- DSA COPY
- chatgpt
- Hamrocsit.